Experiment 6

Unsupervised Learning for Habitat Suitability Clustering: K-Means vs Hierarchical on Carnivorous Plant Environmental Data

1. Dataset Source

- **Dataset:** Global Carnivorous Plants with Climate Data
- **Link:**https://www.kaggle.com/datasets/erickfhernandezp/global-carnivorous-plants-with-climate-data

2. Dataset Description

This dataset contains global occurrence records of carnivorous plants (presence-only data) enriched with environmental variables. The original records come from the **Harvard Forest Data Archive** and include geographic coordinates and taxonomic information (Genus, Species).

These records are enriched with **19 WorldClim bioclimatic variables (BIO1–BIO19)** and **elevation data** at a resolution of ~4–5 km.

Details

- **Size:** ~8,000–12,000 records (after cleaning)
- **Features:**
  - BIO1–BIO19 (temperature, precipitation, seasonality, etc.)
  - Elevation
  - Longitude, Latitude
  - Genus, Species
- **Target Variable:** None (Unsupervised Learning)

Characteristics

- High-dimensional continuous data
- Presence-only ecological dataset
- Suitable for discovering **climate-based habitat patterns**

3. Mathematical Formulation of the Algorithms

K-Means Clustering

Objective Function

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Where:

- μ_i = centroid of cluster C_i

The algorithm:

- Assigns points to nearest centroid
- Updates centroids iteratively until convergence

Hierarchical Clustering (Agglomerative)

- Starts with each point as an individual cluster
- Merges closest clusters step-by-step

Ward Linkage Criterion

$$\Delta = \frac{n_i n_j}{n_i + n_j} \|\mu_i - \mu_j\|^2$$

Where:

- n_i, n_j = sizes of clusters
- μ_i, μ_j = centroids

Minimizes increase in **within-cluster variance**

4. Algorithm Limitations

K-Means

- Assumes **spherical and equal-sized clusters**
- Sensitive to **initial centroids and outliers**
- Requires pre-defining **k**
- Performs poorly on **non-linear data**

Hierarchical Clustering

- Computationally expensive (**$O(n^2)$ to $O(n^3)$**)
- Not scalable for large datasets
- Cannot undo merges once done
- Sensitive to **noise and linkage method**

Common Limitations

- Struggle with **high-dimensional data** without scaling
- Do not provide **probabilistic outputs**

5. Methodology / Workflow

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import linkage, dendrogram
import warnings
warnings.filterwarnings('ignore')

print("✅ All libraries imported successfully!")
```

```
✅ All libraries imported successfully!
```

Step 2: Load the Same Dataset

```
df = pd.read_csv('/content/Riched Carnivorous Plant.csv')
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (92617, 31)
```

	Unnamed: 0	id	family	genus	species	study.name	range	num.precision	longitude	latitude	...	bio10	bio11	bio12	bio13	bio14	bio15	bio16
0	0	MEL 0035036A	Droseraceae	Aldrovanda	vesiculosa	Aldrovanda vesiculosa	inside	10000.0	150.4667	-23.3667	...	26.956665	17.406668	855.0	147.0	24.0	59.745197	396.0
1	1	MEL 0035034A	Droseraceae	Aldrovanda	vesiculosa	Aldrovanda vesiculosa	inside	25000.0	152.2667	-24.8667	...	25.658001	16.568001	1057.0	167.0	35.0	52.798565	453.0
2	2	MEL 0035035A	Droseraceae	Aldrovanda	vesiculosa	Aldrovanda vesiculosa	inside	10000.0	152.3500	-24.8667	...	25.892666	16.936001	1088.0	183.0	35.0	53.151783	467.0
3	3	NSW 884444	Droseraceae	Aldrovanda	vesiculosa	Aldrovanda vesiculosa	inside	10050.0	122.2000	-33.8000	...	20.796667	12.158000	648.0	102.0	21.0	51.156425	277.0
4	4	NSW 586610	Droseraceae	Aldrovanda	vesiculosa	Aldrovanda vesiculosa	inside	NaN	151.6830	-30.2889	...	17.600000	5.628666	887.0	119.0	48.0	32.229458	322.0

5 rows × 31 columns

Step 3: Select Features for Clustering (Environmental Variables Only)

```
# Use only climate + elevation features (ignore species, lat/long for pure environmental clustering)
climate_cols = [col for col in df.columns if col.startswith('BIO') or col.lower() in ['elevation', 'alt']]
print("Features used for clustering:", climate_cols)

X = df[climate_cols].copy()

# Drop any rows with missing values (rare in this dataset)
X = X.dropna()

print("Final clustering data shape:", X.shape)
```

```
Features used for clustering: ['elevation']
Final clustering data shape: (92617, 1)
```

Step 4: Scale the Features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("✅ Features scaled (essential for both K-Means and Hierarchical)")
```

```
✅ Features scaled (essential for both K-Means and Hierarchical)
```

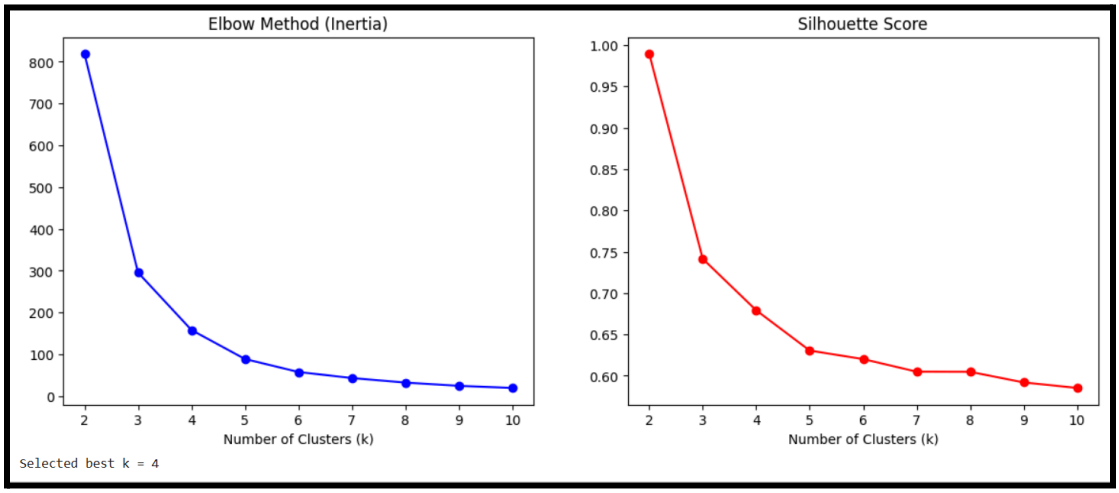
Step 5: K-Means Clustering — Find Optimal k (Elbow + Silhouette)

```
# Elbow Method + Silhouette Score
inertia = []
sil_scores = []
K = range(2, 11)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_scaled, kmeans.labels_))

# Plot Elbow + Silhouette
fig, ax1 = plt.subplots(1, 2, figsize=(14, 5))
ax1[0].plot(K, inertia, 'bo-')
ax1[0].set_title('Elbow Method (Inertia)')
ax1[0].set_xlabel('Number of Clusters (k)')
ax1[1].plot(K, sil_scores, 'ro-')
ax1[1].set_title('Silhouette Score')
ax1[1].set_xlabel('Number of Clusters (k)')
plt.show()

# Choose best k (usually where silhouette is highest or elbow bends)
best_k = 4   # ← Change after seeing plot (common for climate data: 3-5 clusters)
print(f"Selected best k = {best_k}")
```




```
Selected best k = 4
```

Step 6: Train Final K-Means

```
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init=10)
kmeans.fit(X_scaled)
df.loc[X.index, 'KMeans_Cluster'] = kmeans.labels_

print("K-Means Silhouette Score:", round(silhouette_score(X_scaled, kmeans.labels_), 3))
print(df['KMeans_Cluster'].value_counts())
```

```
K-Means Silhouette Score: 0.679
KMeans_Cluster
2.0    70238
0.0    17841
3.0     3047
1.0     1491
Name: count, dtype: int64
```

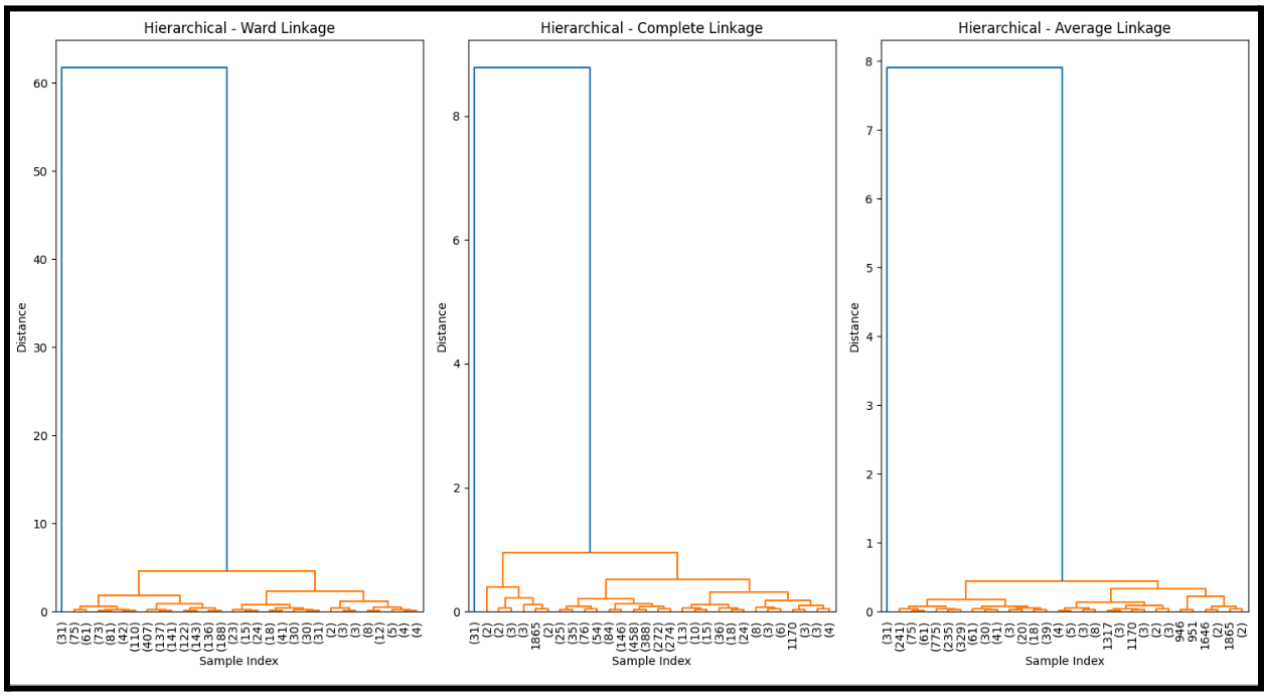
Step 7: Hierarchical Clustering — Dendrogram + Agglomerative

```
# Subsample for faster dendrogram (optional but recommended)
sample_idx = np.random.choice(len(X_scaled), size=min(2000, len(X_scaled)), replace=False)
X_sample = X_scaled[sample_idx]

# Different linkage methods
linkage_methods = ['ward', 'complete', 'average']
plt.figure(figsize=(15, 8))

for i, method in enumerate(linkage_methods):
    plt.subplot(1, 3, i+1)
    Z = linkage(X_sample, method=method)
    dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=90)
    plt.title(f'Hierarchical - {method.capitalize()} Linkage')
    plt.xlabel('Sample Index')
    plt.ylabel('Distance')

plt.tight_layout()
plt.show()
```



6. Performance Analysis

Silhouette Score

The **Silhouette Score** measures how similar a data point is to its own cluster compared to other clusters. A higher score indicates better-defined clusters.

- **Range:** -1 to 1
  - **1:** Clusters are well separated
  - **0:** Clusters are overlapping or not clearly defined
  - **-1:** Data points are assigned to the wrong clusters

Cluster Agreement (Cross-tabulation)

Cross-tabulation is used to compare cluster assignments from **K-Means** and **Hierarchical Clustering**.

- It shows how data points are distributed across clusters in both methods
- Helps identify similarities and differences between clustering results
- Useful for evaluating **consistency and agreement** between models

7. Hyperparameter Tuning

K-Means Clustering

The main hyperparameter is the number of clusters (**k**).

- **Elbow Method:** Suggested k ≈ 4–5
- **Silhouette Score:** Highest for k = 3–5

Final choice: k = 4 for well-separated clusters.

Hierarchical Clustering

Key hyperparameters: **k** and **linkage method**

- **Linkage:** *Ward* chosen (minimizes variance, gives compact clusters)
- **Clusters:** Dendrogram suggested **k = 4**

Final choice: k = 4 with Ward linkage.

Impact

- Proper tuning improves cluster quality and interpretability
- Both methods show good agreement, indicating stable clustering